

Agenda

- ① Prototype Design Pattern
- ② Registry Design Pattern

Prototype Design Pattern

↳ First version

Our fav phone companies first create
a few prototypes before mass productⁿ.

↳ these will be expensive

When phone company mass produces, 99.9%
attrs will be the same (camera, screen,
speakers) but some attrs will be unique
(imei_id, apple's unique id etc)

We have an object that requires an
api call or a DB call to be created.

expensive (space & time)

We want multiple replicas of this object.

We will use prototype design pattern to create the first version & then we will replicate the same obj to create copy objects

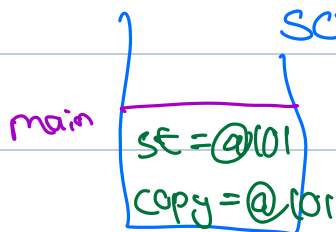
Implementation

Approach # 1 :
main() {

Student st = getStudent();

Student copy = st;

copy.name = 'ABC';
cout << st.name; ←



Stack



Heap

Cons: We are not actually copying here.

Approach #2: Copy attrs manually

```
main() {
```

```
    Student st = getStudent();
```

Student

↑

Intelligent
Student

```
    Student copy = new Student();
```

```
    copy.name = st.name;
```

```
    copy.age = st.age;
```

```
    copy.univ = st.univ;
```

```
    :  
    :  
    :  
    :
```

Pros: ① Will work in most cases

Cons: ① Every we need to copy, we will need to duplicate the code.

② New attrs handling is difficult.

③ Private attrs cannot be

Copied

① SRP & OCP violation.

```
Student st → getStudent();
```

```
if (st instanceof Student) {
```

```
    Student copy = new Student();
```

```
    copy.name = st.name;
```

```
    .  
    .  
    .
```

```
}
```

```
else if (st instanceof IS) {
```

```
    .  
    .  
    .  
    .
```

```
}
```

SRP X
OCP X

Approach #3 : Copy constructor

```
class Student {  
    private int age;  
    public Student (Student og) {  
        this.name = og.name;  
        this.age = og.age;  
        .  
        .  
        .  
        this.address = og.address;  
    }  
}
```

}

Main :

```
Student st = getStudent();
```

```
Student copy = new Student  
                (st);
```

if (st instanceof Student) {

SRP ✗

OCP ✗

copy = new Student(st);

}

else if (

copy = new IS(st);

}

Pros: ① New attrs, easy to manage.

② Private attrs can be copied

③ No code duplication

Cons: ① SRP & OCP violation

```
Bird b = getBird();  
b.fly();
```

↑ method overriding

Approach #1: clone / copy method

```
class Student {
```

```
    public Student (Student st) {  
        this.name = st.name;  
        :  
        :  
    }
```

}

```
    public Student clone () {  
        return new Student (this);  
    }
```

}

3

Main

```
Student s = getStudent();
```

```
Student copy = s.clone();
```

```
class IntelligentStudent {  
    int iq;  
    // extends Student
```

```
    public IS (IS is) {  
        super (is);  
        this.iq = is.iq;  
    }
```

```
}
```

```
    public IntelligentStudent  
        clone () {  
        return new IS (this);  
    }
```

3

3

Student
↓ ↙ Interface

Student s = getStudent();

Student copy = s.clone();

Cons: No cons

Pros: ① No duplicatⁿ

② New obj

copy

③ Private attr ✓

④ SRP & OCP ✓

SRP & OCP violation → Method
Overriding

Cloneable interface in java can be
used.

Real life use case:

Query : {
going to ^{change} query : "Find me restaurants";
const {
 "openai-api-key" : "sk -";
 "url" : "chat.openai.com/query";
}

3

Query q = getPrototypeQuery();

Query cloned = q.clone();

cloned.setQuery("Get input from user");

sort(cloned.execute());

Break for 5 mins: 8:22 AM

↳ Registry DP

↳ Implementation

Registry Design Pattern

↳ Register: is basically holding certain data

Registry design pattern: holds objects for you.

Any object that you don't want to use right away, but you will require it in future, you should use registry design pattern.

Registry is used with Prototype to store prototypes.

Has 2 methods:

① ^{void} register (String key, Student s)

↳ store the given obj inside registry.

② Student get (String key)

↳ is to return the obj with the given key.

Registry uses map internally.

Registry should always be singleton.